

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ĐỒ ÁN MÔN CÁC HỆ THỐNG PHÂN TÁN

Đề tài: Xây dựng ứng dụng chat P2P bằng Java

Nhóm: 19

Giảng viên hướng dẫn: Kim Ngọc Bách

Thành viên nhóm:

Nguyễn Xuân Khương – B22DCCN474

Nguyễn Việt Tiến – B22DCCN726

Nguyễn Văn Quân - B22DCCN666

Hà Nội, 2026

MỤC LỤC

MỤC LỤC	2
DANH MỤC HÌNH ẢNH	3
I. GIỚI THIỆU ĐỀ TÀI	4
1.1. Mục tiêu đề tài	4
1.2. Kiến trúc và công nghệ sử dụng	5
II. KIẾN TRÚC HỆ THỐNG	7
2.1. Các thành phần chính trong hệ thống	7
2.2. Luồng hoạt động tổng quát của hệ thống	8
2.3. Mô hình lưu trữ dữ liệu	9
III. GIAO THỨC TRAO ĐỔI THÔNG ĐIỆP	10
3.1. Giao thức giao tiếp giữa Peer và Bootstrap Server	10
3.2. Giao thức giao tiếp trực tiếp giữa các Peer	11
3.3. Cơ chế mã hóa nội dung thông điệp	12
3.4. Luồng xử lý và gửi tin nhắn riêng trực tiếp	13
3.5. Luồng xử lý tin nhắn nhóm và quảng bá (Broadcast)	13
IV. CƠ CHẾ PHÁT HIỆN NÚT MẠNG (PEER DISCOVERY)	15
4.1. Định vị máy chủ gốc bằng cơ chế UDP Broadcast	15
4.2. Duy trì trạng thái trực tuyến bằng cơ chế Nhịp tim (Heartbeat) ...	16
4.3. Cơ chế tự động giải phóng nút mạng ngoại tuyến (Timeout)	17
V. XỬ LÝ LỖI VÀ THỬ NGHIỆM HỆ THỐNG	18
5.1. Cơ chế phòng vệ và xử lý ngoại lệ trong mã nguồn	18
5.2. Kịch bản thử nghiệm thực tế hệ thống	19
VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	29
6.1. Ưu điểm của hệ thống	29
6.2. Hạn chế còn tồn tại	29
6.3. Hướng phát triển và nâng cấp đề xuất	30

DANH MỤC HÌNH ẢNH

Hình 1.1: Giao diện đăng nhập / đăng ký	4
Hình 2.1: Sơ đồ kiến trúc tổng thể của hệ thống	7
Hình 2.2 : Luồng hoạt động tổng quát của hệ thống	9
Hình 3.1 : Terminal khi chạy Bootstrap Server	11
Hình 3.2 : Terminal khi chạy Web App	11
Hình 3.3 : Giao diện khi Peer nhận tin	13
Hình 3.4 : Giao diện khi Peer thực hiện broadcast message	14
Hình 3.1 : Quy trình Peer Discovery và định vị Bootstrap Server bằng UDP Broadcast	16
Hình 5.1 : Quá trình biên dịch và đóng gói thành công hệ thống bằng Maven ..	19
Hình 5.2 : Khởi động máy chủ trung tâm Bootstrap Server	19
Hình 5.3 : Khởi tạo nhiều Peer Node trên các cổng dịch vụ khác nhau	20
Hình 5.4.1 : Quá trình đăng ký tài khoản	21
Hình 5.4.2 : Lưu trữ thông tin user vào file users.json	22
Hình 5.5 : Cơ chế Khám phá nút mạng (Peer Discovery)	22
Hình 5.6.1 : Tính năng trò chuyện riêng trực tiếp bên A	23
Hình 5.6.2 : Tính năng trò chuyện riêng trực tiếp bên B	23
Hình 5.7 : Tính năng thiết lập và trò chuyện nhóm	23
Hình 5.8.1 : Peer A gửi 1 thông điệp Broadcast	24
Hình 5.9.1 : Terminal khi Peer A gửi tin nhắn lúc Peer B offline	25
Hình 5.9.2 : Giao diện Peer A gửi tin nhắn lúc Peer B offline	25
Hình 5.9.4 : Giao diện Peer B nhận được tin nhắn của Peer A lúc online	25
Hình 5.10 : Giao diện tính năng truyền tải tệp tin trực tiếp	26
Hình 5.11.1 : Folder nơi lưu trữ lịch sử trò chuyện	27
Hình 5.11.2 : Cấu trúc lưu trữ lịch sử tin nhắn cục bộ dạng văn bản	27
Hình 5.11.3 : Giao diện lịch sử trò chuyện	28

I. GIỚI THIỆU ĐỀ TÀI

Trong bối cảnh các hệ thống phân tán ngày càng phát triển, mô hình mạng ngang hàng (Peer-to-Peer - P2P) đã khẳng định được ưu thế lớn nhờ khả năng giảm tải cho máy chủ trung tâm, tối ưu hóa băng thông và tăng cường tính chịu lỗi. Nhằm nghiên cứu sâu hơn về cơ chế vận hành của kiến trúc này, đề án tập trung xây dựng một **Ứng dụng trò chuyện ngang hàng (P2P Chat Application)** sử dụng ngôn ngữ lập trình Java.

Hệ thống cho phép các nút mạng (Peers) kết nối trực tiếp để thực hiện các chức năng: đăng ký/đăng nhập tài khoản, tự động phát hiện các Peer đang hoạt động (Peer Discovery), gửi tin nhắn riêng (Private Chat), nhắn tin nhóm (Group Chat), gửi tin nhắn quảng bá (Broadcast) và truyền tải tập tin trực tiếp giữa các nút (Direct File Transfer). Bên cạnh kiến trúc giao tiếp ngang hàng cốt lõi, hệ thống tích hợp một **Bootstrap Server** đóng vai trò là Centralized Tracker. Thành phần này hỗ trợ quản lý trạng thái, định tuyến trong giai đoạn khởi tạo kết nối và lưu trữ tin nhắn tạm thời (Pending Messages) khi người nhận ở trạng thái ngoại tuyến (Offline).



The image shows a web interface for a P2P Chat application. At the top, there is a logo with a speech bubble icon and the text "P2P Chat". Below the logo, the text "Hệ thống chat phân tán có mã hoá" (Decentralized encrypted chat system) is displayed. The interface features two main buttons: "Đăng nhập" (Login) in blue and "Đăng ký" (Register) in grey. Below these buttons are two input fields: "Tên đăng nhập" (Username) and "Mật khẩu" (Password). A large blue button labeled "Đăng nhập" is positioned below the input fields. At the bottom, there is a status bar showing a green circle icon and the text "Bootstrap: 172.16.0.2".

Hình 1.1: Giao diện đăng nhập / đăng ký

1.1. Mục tiêu đề tài

Việc nghiên cứu và triển khai đề án nhằm đạt được các mục tiêu cốt lõi sau:

- **Nghiên cứu lý thuyết:** Hiểu rõ nguyên lý hoạt động của mô hình hệ thống phân tán dạng lai (Hybrid Peer-to-Peer) và vai trò của nút gốc (Bootstrap Server/Tracker) trong việc định danh, quản lý cấu hình mạng.
- **Lập trình mạng socket:** Làm chủ cơ chế truyền tải thông điệp, thiết lập và duy trì các kết nối tin cậy thông qua giao thức TCP Socket giữa các Peer với nhau.
- **Quản lý trạng thái mạng:** Triển khai thành công cơ chế phát hiện nút mạng trực tuyến (Peer Discovery), duy trì kết nối thông qua gói tin nhịp tim (Heartbeat) và xử lý ngắt kết nối khi quá thời gian phản hồi (Timeout).
- **Xử lý dữ liệu và lưu trữ:** Xây dựng cơ chế mã hóa dữ liệu, đồng bộ và lưu trữ lịch sử trò chuyện cục bộ (Local Chat History), đồng thời xử lý hàng đợi tin nhắn chờ trên server khi các nút mạng mất kết nối.
- **Thử nghiệm thực tế:** Đánh giá hiệu năng và độ ổn định của hệ thống thông qua việc giả lập nhiều tài khoản chạy đồng thời trên cùng một thực thể (Localhost) hoặc phân tán trên các thiết bị khác nhau trong cùng mạng cục bộ (LAN).

1.2. Kiến trúc và công nghệ sử dụng

Để hiện thực hóa các mục tiêu phân tán của hệ thống, ứng dụng được xây dựng dựa trên nền tảng ngôn ngữ Java cùng các thư viện hỗ trợ chuyên dụng. Sự kết hợp này đảm bảo tính toàn vẹn của dữ liệu, hiệu năng truyền tải và khả năng mở rộng mạng lưới các nút. Cụ thể bao gồm:

- **Nền tảng phát triển lõi (Java 17 & Maven):** Hệ thống sử dụng phiên bản Java 17 (LTS) làm ngôn ngữ lập trình chủ đạo, tận dụng các cải tiến về hiệu năng hệ thống và quản lý bộ nhớ để xây dựng đồng thời cả Bootstrap Server, logic xử lý cho Peer Node và ứng dụng Web. Chu trình biên dịch, quản lý các thư viện phụ thuộc (dependencies) cũng như đóng gói ứng dụng thành các tệp tin JAR được chuẩn hóa thông qua công cụ quản lý dự án Maven.
- **Giao tiếp và lập trình mạng (TCP Socket & UDP Datagram):** Đây là thành phần xương sống quản lý tầng truyền tải của mạng phân tán. Hệ thống kết hợp linh hoạt cả hai giao thức:
 - *TCP Socket:* Được áp dụng cho luồng giao tiếp chính giữa các Peer với nhau hoặc giữa Peer và Tracker, đảm bảo tin nhắn, tệp tin được truyền tải chính xác, có thứ tự và không bị mất mát gói tin.
 - *UDP Datagram:* Được sử dụng dưới dạng phát quảng bá (broadcast/multicast) không hướng kết nối, giúp các Peer mới tham gia mạng có thể dò quét và tìm kiếm nhanh vị trí của Bootstrap Server trong không gian mạng cục bộ.
- **Tầng dịch vụ Web và Điều phối giao diện (Javalin 6.1.3):** Thay vì sử dụng các framework nặng nề, dự án tích hợp Javalin 6.1.3 — một cấu trúc Web server gọn nhẹ cho Java. Thành phần này đóng vai trò cung cấp hệ thống RESTful API kết nối giữa tầng xử lý logic phía sau (Backend) và giao diện điều khiển của người dùng trên trình duyệt (Frontend).
- **Xử lý định dạng dữ liệu (Jackson Databind):** Để quản lý thông tin trạng thái trong môi trường phân tán mà không phụ thuộc vào hệ quản trị cơ sở dữ liệu công kênh, hệ thống sử dụng thư viện Jackson Databind. Thư viện này hỗ trợ chuyển đổi tuần tự (serialization) và ngược tuần tự (deserialization) cấu trúc dữ liệu JSON để đọc/ghi mượt mà vào các tệp tin cấu hình cục bộ như users.json và pending_messages.json.
- **Cơ chế mã hóa và Bảo mật thông tin:** Yếu tố an toàn thông tin trong mô hình P2P được đảm bảo nghiêm ngặt qua hai lớp bảo vệ:
 - *Mã hóa dữ liệu:* Sử dụng thuật toán mã hóa đối xứng mã nguồn mở **AES** kết hợp chế độ vận hành **CBC** và chuẩn chèn dữ liệu **PKCS5Padding**. Cơ chế này

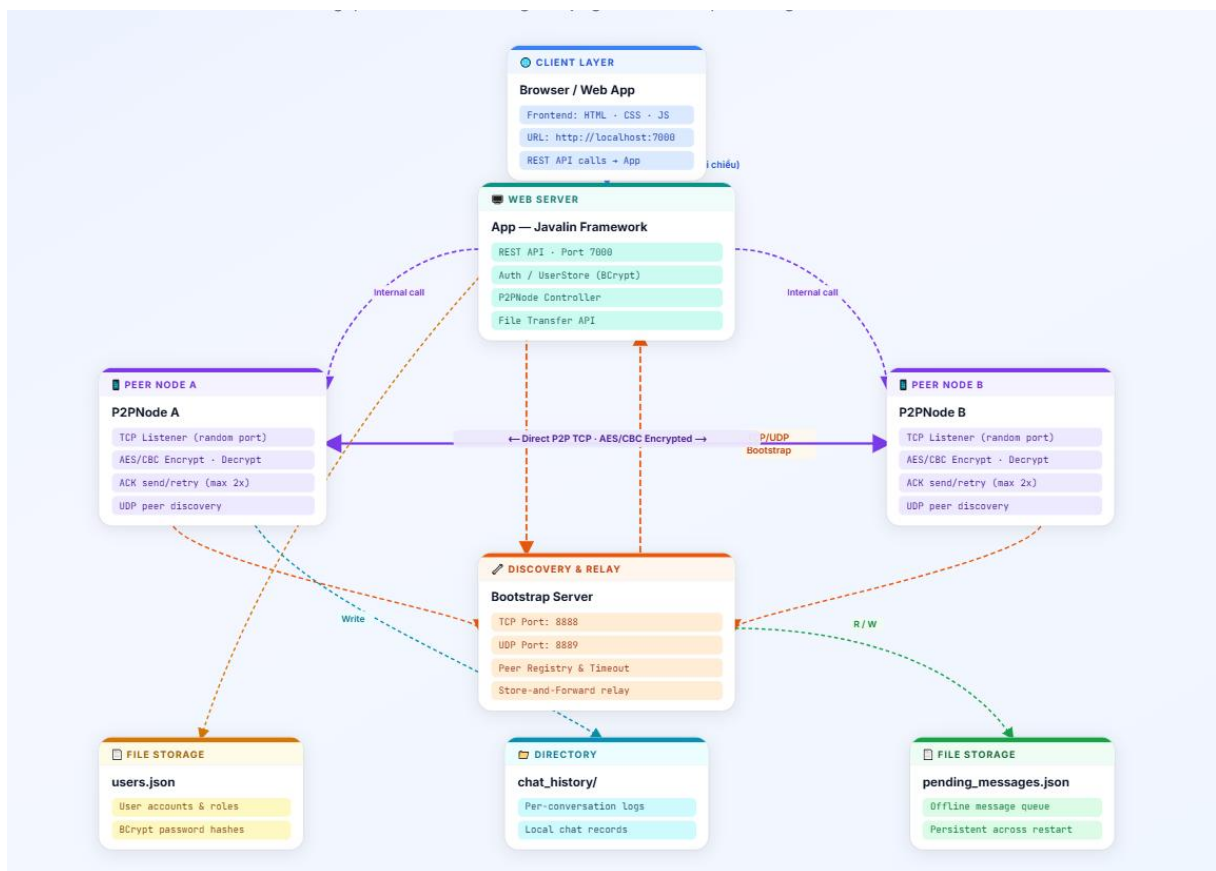
đảm bảo nội dung tin nhắn được chuyển sang dạng bản mã hóa trước khi truyền qua socket, ngăn chặn các hình thức nghe lén trên đường truyền.

- *Bảo mật định danh*: Mật khẩu của người dùng khi thực hiện đăng ký tài khoản sẽ được băm một chiều bằng thuật toán **BCrypt** trước khi lưu trữ, ngăn ngừa tối đa rủi ro lộ lọt thông tin quản trị hệ thống

II. KIẾN TRÚC HỆ THỐNG

Hệ thống được thiết kế dựa trên kiến trúc **Mạng ngang hàng lai (Hybrid Peer-to-Peer)**, kết hợp linh hoạt giữa mô hình Tập trung (Client-Server) và mô hình Phân tán (Pure P2P). Trong cấu trúc này, máy chủ gốc (Bootstrap Server) không tham gia vào quá trình truyền tải dữ liệu cuộc gọi hay nội dung chat chính, mà chỉ đóng vai trò là một "Tracker trung tâm" hỗ trợ các nút mạng trong giai đoạn khởi tạo (Bootstrap), định danh và khám phá lẫn nhau (Peer Discovery).

Khi liên lạc đã được thiết lập, luồng dữ liệu trao đổi (tin nhắn văn bản, tệp tin) sẽ được truyền phát trực tiếp từ Peer nguồn đến Peer đích thông qua các kênh kết nối TCP Socket độc lập. Tại mỗi nút mạng, một Web Application cục bộ được triển khai, đóng vai trò làm giao diện tương tác người dùng (UI) và cung cấp lớp API trung gian để điều phối các tác vụ mạng của nút ngang hàng (P2PNode).



Hình 2.1: Sơ đồ kiến trúc tổng thể của hệ thống

2.1. Các thành phần chính trong hệ thống

Hệ thống được module hóa thành các thành phần chức năng độc lập, phân tách rõ ràng giữa tầng xử lý mạng, tầng lưu trữ và tầng giao diện:

- **Lớp điều phối trung tâm (Bootstrap Server - BootstrapServer.java):** Vận hành độc lập tại cổng dịch vụ cố định (mặc định là TCP 8888 và UDP 8889). Thành phần này chịu trách nhiệm lắng nghe tín hiệu thăm dò từ các nút mạng mới, quản lý bộ nhớ

đệm chứa danh sách các Peer đang trực tuyến (Online Peer List), đồng thời vận hành hàng đợi tin nhắn chờ để bàn giao dữ liệu khi các nút ngoại tuyến quay trở lại mạng.

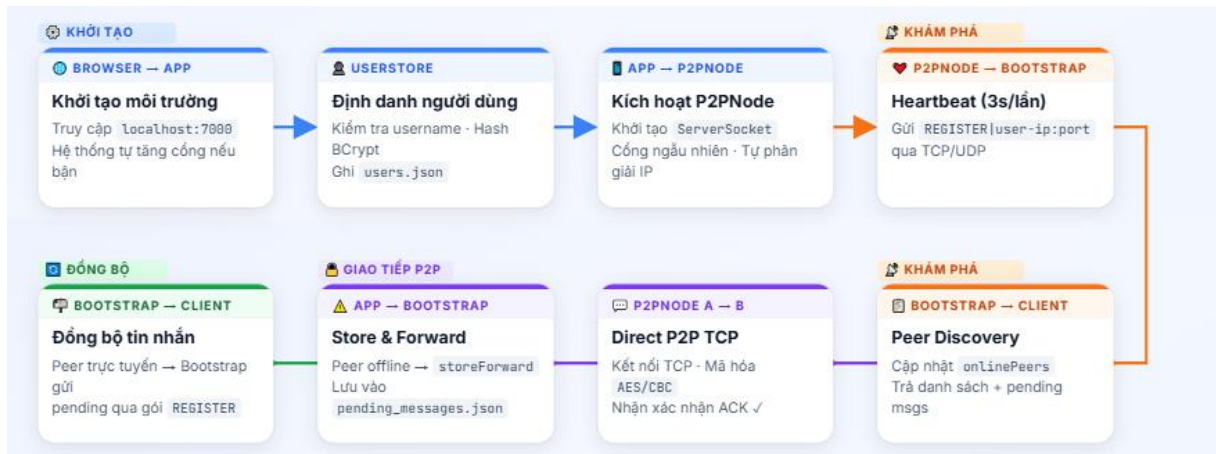
- **Lớp ứng dụng Web (Web App - App.java):** Chạy cục bộ trên mỗi máy trạm thông qua Framework Javalin. Lớp này đảm nhận việc khởi động môi trường nhúng, định tuyến toàn bộ các yêu cầu HTTP REST API từ Frontend (như đăng ký, đăng nhập, lệnh gửi tin nhắn, truyền tệp tin, truy xuất lịch sử) và chuyển tiếp xuống tầng giao tiếp ngang hàng xử lý.
- **Lớp mạng ngang hàng lõi (Peer Node - P2PNode.java):** Trái tim của kiến trúc phân tán. Khi kích hoạt, mỗi Node sẽ tự động mở một cổng ServerSocket ngẫu nhiên còn trống trên hệ thống để chuẩn bị nhận kết nối. Thành phần này trực tiếp quản lý các luồng dữ liệu (Thread) gửi/nhận thông điệp, thực thi các thuật toán mã hóa/giải mã AES, trực tiếp truyền tải file và tương tác với tầng lưu trữ cục bộ.
- **Lớp quản lý định danh (User Store - UserStore.java):** Thực hiện chức năng quản trị tài khoản người dùng tại Client. Thành phần này kiểm tra tính hợp lệ của thông tin đăng ký, thực hiện băm bảo mật mật khẩu bằng cơ chế mã hóa BCrypt và đồng bộ dữ liệu xuống tệp tin cấu hình hệ thống users.json.
- **Lớp quản lý lịch sử (Chat History Store - ChatHistoryStore.java):** Chịu trách nhiệm thực thi các tác vụ I/O (Đọc/Ghi) dữ liệu cuộc hội thoại. Hệ thống phân tách lịch sử theo từng tài khoản đăng nhập và theo từng định danh cuộc trò chuyện riêng biệt để đảm bảo tính an toàn dữ liệu cục bộ.
- **Giao diện người dùng (Frontend - index.html, style.css):** Lớp giao diện trực quan xây dựng trên nền tảng Web tiêu chuẩn, cung cấp các màn hình chức năng bao gồm: biểu mẫu xác thực, bảng giám sát danh sách peer trực tuyến, khung hội thoại thời gian thực, quản lý kênh chat nhóm và trình điều khiển tải lên/tải xuống tệp tin.

2.2. Luồng hoạt động tổng quát của hệ thống

Quá trình vận hành, thiết lập kết nối và trao đổi thông điệp trong mạng được thực hiện tuần tự qua các bước sau:

1. **Khởi tạo môi trường:** Người dùng truy cập vào giao diện ứng dụng Web cục bộ thông qua trình duyệt tại địa chỉ `http://localhost:7000` (hoặc hệ thống tự động tăng tiến cổng nếu cổng 7000 đã bị chiếm dụng).
 2. **Định danh người dùng:** Người dùng tiến hành đăng ký tài khoản. UserStore tiếp nhận, kiểm tra tính duy nhất của Username, tiến hành băm mật khẩu qua BCrypt và ghi nhận vào cơ sở dữ liệu tệp tin users.json.
 3. **Kích hoạt Nút mạng (Node Activation):** Khi đăng nhập thành công, thành phần App sẽ khởi tạo một thực thể P2PNode độc lập. Node này tự động cấp phát một ServerSocket trên cổng ngẫu nhiên và tự động phân giải cấu hình địa chỉ IP nội bộ của thiết bị.
 4. **Duy trì trạng thái trực tuyến (Heartbeat Mechanism):** Ứng dụng thiết lập một luồng chạy ngầm định kỳ 3 giây một lần gửi gói tin nhịp tim dạng bản tin định thức `REGISTER|username-ip:port` hướng tới Bootstrap Server qua giao thức mạng.
 5. **Cập nhật cấu hình mạng (Peer Discovery):** Sau khi nhận tín hiệu, Bootstrap Server xác thực định danh, cập nhật bản đồ các nút mạng đang hoạt động (onlinePeers) và phản hồi lại cho Client danh sách toàn bộ các Peer đang trực tuyến, kèm theo các tin nhắn lưu trữ tạm thời (nếu có).
 6. **Giao tiếp ngang hàng trực tiếp (Direct P2P Messaging):** Khi phát sinh lệnh gửi tin nhắn riêng, Peer nguồn sẽ trích xuất địa chỉ `ip:port` của Peer đích từ danh sách trực tuyến và thiết lập một kết nối TCP Socket trực tiếp. Nội dung tin nhắn sau khi mã hóa sẽ được đẩy qua đường truyền này. Nếu nhận được tín hiệu phản hồi xác nhận (ACK - Acknowledgment) từ đích, tin nhắn được đánh dấu gửi thành công.
-

7. **Xử lý ngoại lệ lưu chuyển (Store and Forward):** Trong trường hợp kết nối TCP trực tiếp đến Peer đích thất bại (Peer đã offline đột ngột hoặc lỗi định tuyến), ứng dụng Web sẽ kích hoạt cơ chế thiết lập cuộc gọi dự phòng (storeForward), đẩy bản tin này ngược lên lưu trữ tạm thời trên bộ nhớ của Bootstrap Server.
8. **Đồng bộ tin nhắn chờ:** Ở chu kỳ kết nối tiếp theo khi Peer nhận chuyển sang trạng thái trực tuyến, Bootstrap Server sẽ tự động đóng gói toàn bộ các tin nhắn chờ này vào gói tin phản hồi của tiến trình REGISTER để cập nhật lại cho Client.



Hình 2.2 : Luồng hoạt động tổng quát của hệ thống

2.3. Mô hình lưu trữ dữ liệu

Hệ thống sử dụng cơ chế cấu trúc tệp tin phẳng (Flat Files) và bộ nhớ đệm RAM để tối ưu hóa tốc độ xử lý mà không cần triển khai hệ quản trị cơ sở dữ liệu quan hệ phức tạp:

- **Tệp tin định danh (users.json):** Lưu trữ cấu trúc dữ liệu của toàn bộ tài khoản hệ thống bao gồm các trường thông tin: tên định danh (username), chuỗi mật khẩu đã băm (passwordHash) và mốc thời gian thiết lập tài khoản (createdAt).
- **Tệp tin hàng đợi ngoại tuyến (pending_messages.json):** Tổ chức lưu trữ cấu trúc các tin nhắn chưa được chuyển phát, phân loại theo từ khóa username của người nhận. Dữ liệu này sẽ được giải phóng (xóa bỏ khỏi tệp) ngay sau khi chuyển phát thành công cho nút mạng online trở lại.
- **Thư mục lưu trữ lịch sử (chat_history/):** Mỗi cuộc hội thoại giữa người dùng hiện tại và một đối tác khác (hoặc một nhóm) sẽ được ghi lại thành một tệp tin văn bản độc lập có định dạng history_<username>_<conversationKey>.txt. Để tối ưu hóa cấu trúc đọc, nội dung tệp tin được phân rã theo cú pháp phân tách bằng ký tự đứng: ME|nội dung_tin_nhắn (đối với tin nhắn gửi đi) hoặc THEM|nội dung_tin_nhắn (đối với tin nhắn nhận về).
- **Quản lý truyền tải tệp tin (File Buffer):** Do đặc thù truyền tải bất đồng bộ, các tệp tin khi được chuyển dịch hoàn tất giữa các cặp Peer thông qua Socket sẽ được nạp tạm thời vào bộ nhớ RAM của Web App dưới dạng cấu trúc bản đồ ánh xạ receivedFiles (chứa mảng byte dữ liệu) và receivedFileNames (chứa tên tệp tương ứng). Sau đó, Frontend sẽ gọi tới API đầu cuối /api/download để người dùng tải tệp tin về thiết bị lưu trữ vật lý một cách an toàn.

III. GIAO THỨC TRAO ĐỔI THÔNG điệp

Hệ thống sử dụng một giao thức hướng văn bản tinh giản (Text-based Protocol) để tối ưu hóa tốc độ đóng gói và phân rã gói tin. Mỗi thông điệp trao đổi qua Socket cấu trúc dưới dạng một dòng văn bản đơn lẻ, trong đó các trường dữ liệu thành phần (Fields) được phân tách nghiêm ngặt bởi ký tự gạch đứng (|).

Tại các nút mạng (P2PNode) và máy chủ gốc (Bootstrap Server), luồng dữ liệu vào/ra được xử lý bất đồng bộ thông qua các lớp chuẩn của Java I/O: sử dụng `BufferedReader.readLine()` để bao bọc và đọc từng dòng dữ liệu từ luồng vào, và `PrintWriter.println()` để đẩy dữ liệu phản hồi xuống luồng ra của Socket kèm ký tự xuống dòng để kết thúc gói tin.

3.1. Giao thức giao tiếp giữa Peer và Bootstrap Server

Tương tác giữa các nút mạng ngang hàng và máy chủ định danh trung tâm được chuẩn hóa qua tập lệnh thủ tục sau:

- **Lệnh dò quét hệ thống (DISCOVER_BOOTSTRAP):**

Phát đi từ ứng dụng Web Client dưới dạng gói tin UDP Broadcast hướng tới cổng 8889 trong toàn bộ phân mạng LAN nhằm tự động định vị địa chỉ IP vật lý của máy chủ Tracker.

- **Lệnh phản hồi định vị (BOOTSTRAP_HERE|ip):**

Phản hồi đơn hướng (Unicast) từ Bootstrap Server ngược về Client để xác nhận trạng thái hoạt động và cung cấp địa chỉ IP tĩnh phục vụ các kết nối TCP tiếp theo.

- **Lệnh đăng ký và duy trì kết nối (REGISTER|username-ip:port):**

Đẩy từ Client tới cổng TCP 8888 của Bootstrap Server. Bản tin này vừa đóng vai trò xác thực phiên làm việc lúc đăng nhập, vừa là gói tin nhịp tim (Heartbeat) cập nhật tọa độ mạng của Peer theo chu kỳ.

- **Lệnh đồng bộ trạng thái (LIST|peer1,peer2,...|PENDING|msg1~~msg2):**

Bản tin phản hồi từ Bootstrap Server sau lệnh REGISTER. Gói tin này chứa danh sách toàn bộ các nút đang trực tuyến (cấu trúc phân tách dấu phẩy) và danh sách các tin nhắn chờ lưu trữ trong hàng đợi ngoại tuyến (phân tách bởi ký tự ~~).

- **Lệnh chuyển tiếp lưu trữ (STORE|recipient|message):**

Gửi từ Peer nguồn lên Bootstrap Server khi tiến trình thiết lập kết nối trực tiếp đến Peer đích bị gián đoạn, yêu cầu máy chủ kích hoạt cơ chế lưu chuyển dự phòng (Store-and-Forward).

- **Lệnh xác nhận lưu trữ (STORED):**

Thông điệp phản hồi từ Bootstrap Server trả về Peer nguồn, xác nhận dữ liệu ngoại tuyến đã được đưa vào hàng đợi an toàn.

- **Lệnh giải phóng nút mạng (LEAVE|username):**

Phát đi từ Client lên Bootstrap Server khi người dùng thực hiện thao tác đăng xuất (Logout), yêu cầu hệ thống xóa bỏ định danh khỏi danh sách trực tuyến ngay lập tức.

```
Peer online: quan1 @ 192.168.133.1:49487
>>> Peer timeout, ?? xóa: quan
Peer online: quan1 @ 192.168.133.1:49487
Peer online: quan1 @ 192.168.133.1:49487
Peer online: quan1 @ 192.168.133.1:49487
Peer online: quan1 @ 192.168.133.1:49487
Peer online: quan1 @ 192.168.133.1:49487
Peer online: quan1 @ 192.168.133.1:49487
Peer online: quan1 @ 192.168.133.1:49487
Peer online: quan1 @ 192.168.133.1:49487
□
```

Hình 3.1 : Terminal khi chạy Bootstrap Server

```
PS C:\DATA\http\p2pChat> java -jar target/p2p-app.jar

https://javalin.io/documentation

[main] INFO io.javalin.Javalin - Javalin started in 974ms \o/
[main] INFO io.javalin.Javalin - Static file handler added: StaticFileConfig(hostedPath=/, d
l=max-age=0}, skipFileFunction=Function1<jakarta.servlet.http.HttpServletRequest, java.lang.I
ar!/public'
[main] INFO io.javalin.Javalin - Listening on http://localhost:7000/
[main] INFO io.javalin.Javalin - You are running Javalin 6.1.3 (released March 2, 2024. Your

=====
>>> Giao di?n chat: http://localhost:7000
=====
```

Hình 3.2 : Terminal khi chạy Web App

3.2. Giao thức giao tiếp trực tiếp giữa các Peer

Khi thực hiện giao tiếp ngang hàng (Pure P2P), các nút thiết lập kết nối TCP Socket trực tiếp và trao đổi theo các định thức cấu trúc sau:

- **Định thức Tin nhắn riêng (Private Message):**

[Private]|senderName-senderAddr|encryptedMsg

Áp dụng cho các hội thoại đôi. Nội dung thông điệp gốc (msg) bắt buộc phải trải qua tiến trình mã hóa mật mã đối xứng AES trước khi đẩy ra Socket đường truyền.

- **Định thức Tin nhắn nhóm (Group Message):**

[Group]|groupId|senderName-senderAddr|encryptedMsg

Sử dụng để định tuyến thông điệp trong một không gian hội thoại chung. Gói tin bổ sung định danh nhóm (groupId) để các Peer nhận diện và phân loại luồng giao diện.

- **Định thức Tin nhắn quảng bá (Broadcast Message):**

[Broadcast]|encryptedMsg

Chuyển phát một nội dung bản mã đến toàn bộ các nút mạng đang có trạng thái trực tuyến trong danh bạ cấu hình của hệ thống.

- **Định thức Mời tham gia nhóm (Group Invitation):**

[GROUP_INVITE]|groupId|groupName|memberAddrs|sender

Gửi từ nút khởi tạo đến danh sách các nút mục tiêu (memberAddrs), cung cấp đầy đủ thông tin định danh và cấu hình để các Peer tự động thiết lập cấu trúc nhóm cục bộ.

- **Định thức Giải tán nhóm (Group Deletion):**

[GROUP_DELETE]|groupId|groupName|sender

Thông báo phát đi từ quản trị viên nhóm để yêu cầu các thành viên đồng loạt xóa bỏ cấu trúc lưu trữ nhóm trên máy cục bộ.

- **Định thức Truyền tải tệp tin (File Transfer):**

[FILE]|senderName-senderAddr|fileName|base64data

Do đặc thù Socket hướng văn bản, toàn bộ dữ liệu nhị phân (Binary) của tệp tin sẽ được chuyển đổi mã hóa sang chuỗi ký tự an toàn thông qua chuẩn mã hóa Base64 trước khi truyền tải.

- **Định thức Xác nhận gói tin (Acknowledgment):**

ACK

Chuỗi ký tự phản hồi ngắn gọn từ Peer nhận trả ngược về Peer gửi ngay sau khi đọc và kiểm tra tính toàn vẹn của gói tin thành công, phục vụ cơ chế kiểm soát lỗi đường truyền.

3.3. Cơ chế mã hóa nội dung thông điệp

Yếu tố an toàn thông tin dữ liệu trong môi trường mạng phân tán được bảo vệ bằng tầng mật mã đối xứng:

Trong thực thể P2PNode, toàn bộ nội dung văn bản thuần (Plaintext) trước khi gửi đi đều phải trải qua tiến trình mã hóa bằng thuật toán AES vận hành ở chế độ mã khối CBC (Cipher Block Chaining) với chuẩn chèn dữ liệu PKCS5Padding. Khóa mật mã sử dụng là một chuỗi cố định có độ dài 16 byte (P2PChatSecretKey).

Để triệt tiêu khả năng bị tấn công phân tích tần suất bản mã, mỗi tiến trình mã hóa sẽ tự động sinh một Vector khởi tạo (IV - Initialization Vector) ngẫu nhiên có độ dài 16 byte

thông qua lớp mã hóa an toàn SecureRandom. Gói tin mã hóa hoàn chỉnh sau đó được đóng gói theo cú pháp định dạng:

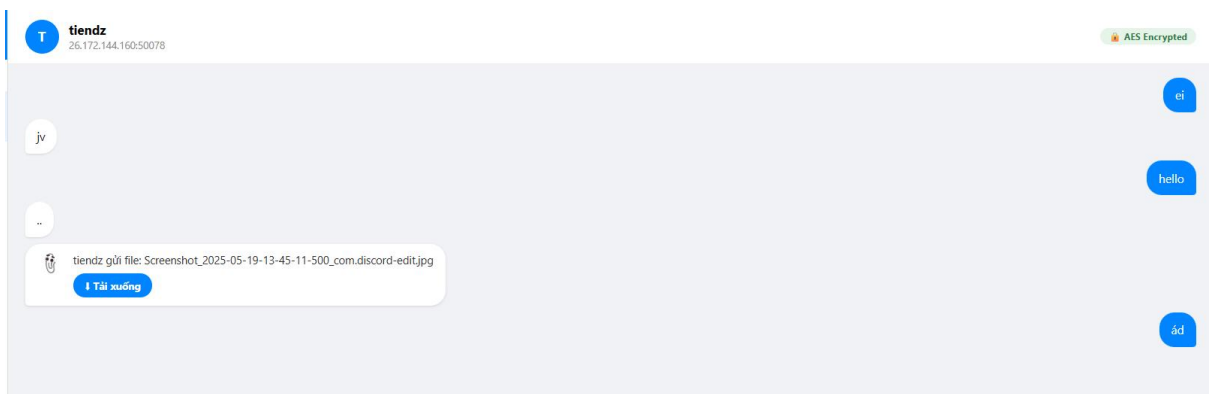
base64(iv) : base64(ciphertext)

Khi tiếp nhận chuỗi ký tự này qua đường truyền, Peer nhận sẽ thực hiện bóc tách chuỗi dựa trên ký tự phân tách hai chấm (:), chuyển đổi ngược từ Base64 về mảng Byte dữ liệu gốc, dùng IV để giải mã (decryptMessage()), sau đó mới chuyển tiếp nội dung bản rõ vào hàng đợi hiển thị (messageQueue) và đồng bộ vào bộ nhớ lưu trữ lịch sử.

3.4. Luồng xử lý và gửi tin nhắn riêng trực tiếp

Quá trình điều phối và truyền tải một tin nhắn riêng giữa hai Peer tuân thủ theo kịch bản tuần tự sau:

1. Người dùng tương tác trên giao diện Web (Frontend), lựa chọn một nút mạng đích trong danh sách và nhập nội dung thông điệp.
2. Frontend thực hiện lệnh gọi bất đồng bộ (Fetch API) tới đầu cuối /api/send của Web App cục bộ, truyền tải các tham số định danh bao gồm: target (tọa độ mạng), targetName (tên người nhận) và msg (nội dung gốc).
3. Lớp điều phối App tiếp nhận yêu cầu và gọi phương thức xử lý mạng lớp dưới: node.sendDirect(targetAddr, targetName, msg).
4. Thực thể P2PNode kích hoạt luồng xử lý: gọi hàm mã hóa dữ liệu, đóng gói theo định thức [Private]|sender|encryptedMsg, đồng thời khởi tạo một kết nối TCP Socket hướng thẳng tới tọa độ mạng ip:port của nút nhận.
5. Tại đầu nhận, luồng lắng nghe (Listening Thread) đọc dòng dữ liệu thô, gọi hàm decryptMessage(), đẩy dữ liệu bản rõ vào messageQueue để Frontend cập nhật lên màn hình, ghi nhận vào ChatHistoryStore và lập tức phản hồi chuỗi ACK về đường truyền Socket.
6. Tại đầu gửi, nếu nhận được chuỗi ký tự ACK trong thời gian quy định, hệ thống ghi nhận tin nhắn thành công và lưu lịch sử cục bộ dưới dạng ME|nội_dung_tin_nhắn. Trong trường hợp kết nối thất bại hoặc không nhận được ACK sau 02 lần thử lại (Retries), hệ thống trả về giá trị lỗi (false), điều hướng App chuyển đổi sang luồng xử lý dự phòng Store-and-Forward để gửi lên máy chủ Tracker.

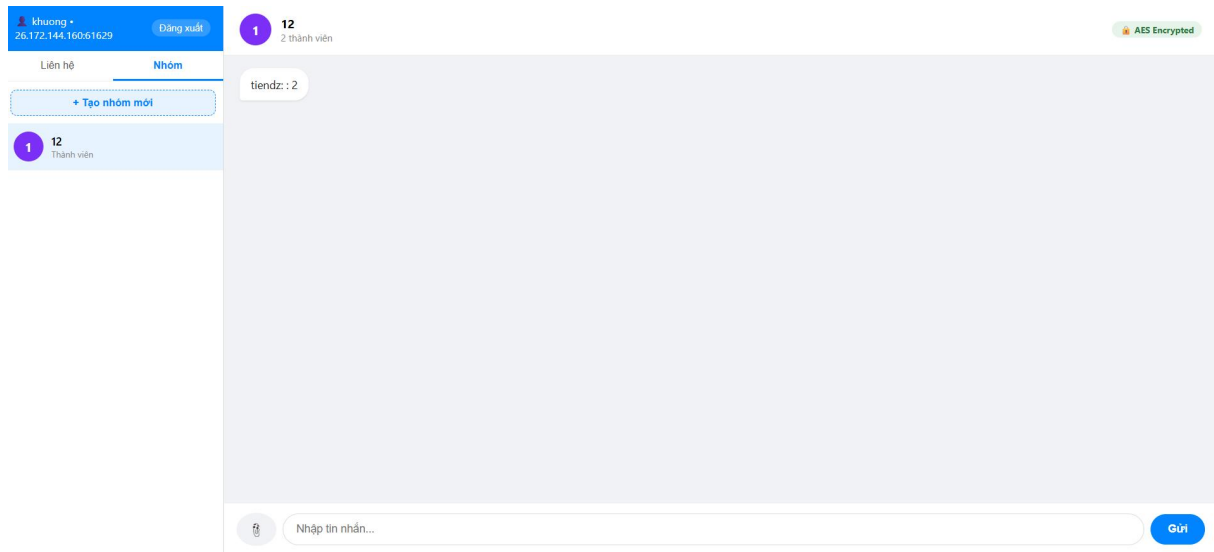


Hình 3.3 : Giao diện khi Peer nhắn tin

3.5. Luồng xử lý tin nhắn nhóm và quảng bá (Broadcast)

Đặc trưng cốt lõi của ứng dụng P2P này là hoàn toàn loại bỏ máy chủ trung tâm trong việc định tuyến tin nhắn đa hướng:

- **Xử lý Trò chuyện nhóm (Group Chat):** Khi một thông điệp nhóm được phát đi, P2PNode không đẩy tin nhắn lên server để phân phát. Thay vào đó, Node này sẽ truy vấn cấu hình nhóm cục bộ để lấy ra danh sách tọa độ mạng của tất cả các thành viên hiện tại. Sau đó, hệ thống gọi hàm `sendToAddr()` để thực hiện một vòng lặp, mở tuần tự các kết nối TCP độc lập và đẩy gói tin dạng [Group] trực tiếp tới từng địa chỉ thành viên.
- **Xử lý Quảng bá (Broadcast):** Đối với tính năng gửi tin nhắn toàn mạng, P2PNode sẽ duyệt qua toàn bộ cấu hình mạng lưu trữ các nút mạng đang trực tuyến (`onlinePeers`) được đồng bộ từ Bootstrap Server. Hệ thống bóc tách trường thông tin địa chỉ `ip:port` của từng Peer (loại trừ chính nó) và thực hiện phân phát song song thông điệp định thức [Broadcast] tới toàn bộ mạng lưới cấu hình.



Hình 3.4 : Giao diện khi Peer thực hiện broadcast message

IV. CƠ CHẾ PHÁT HIỆN NÚT MẠNG (PEER DISCOVERY)

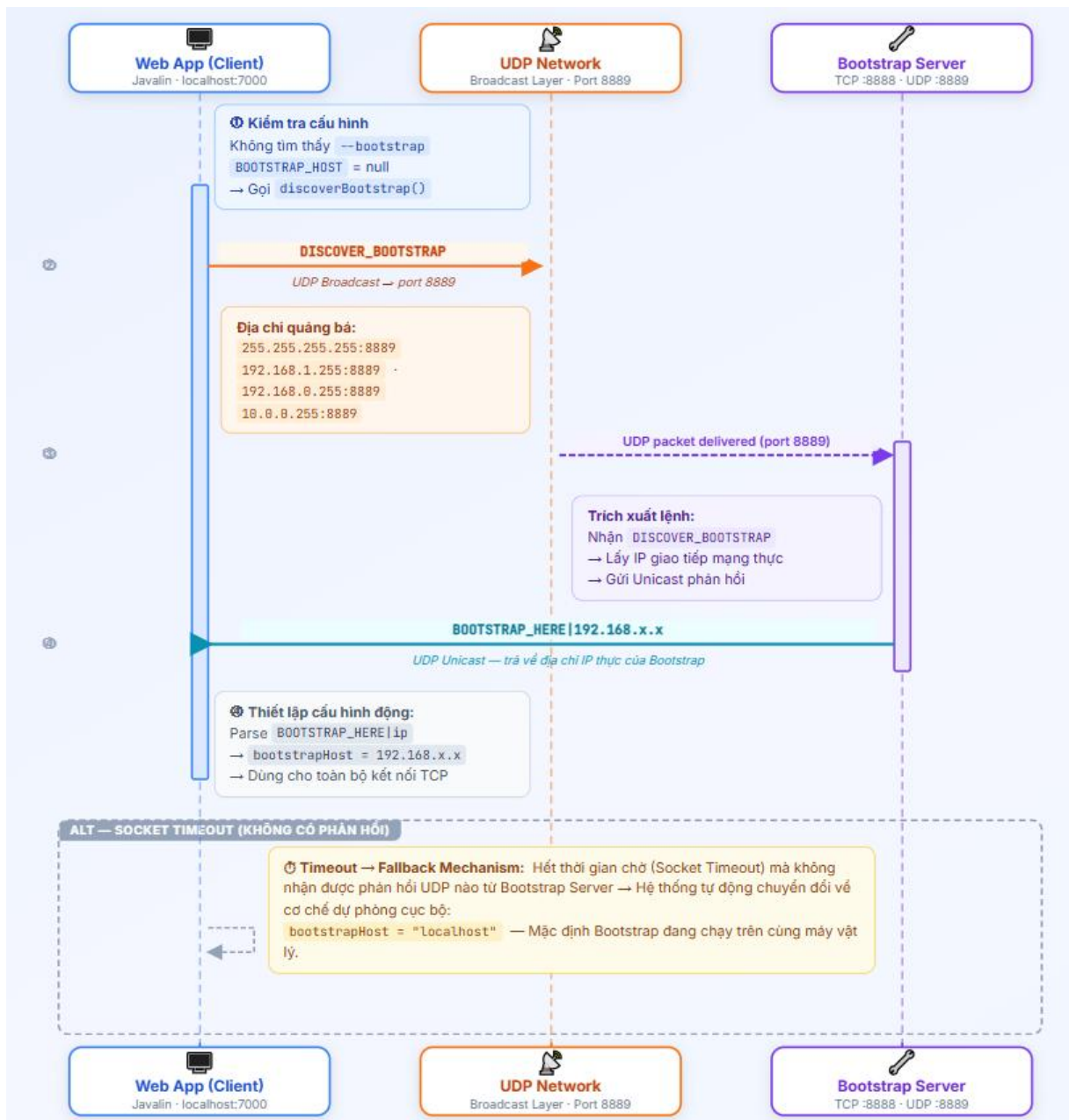
Cơ chế phát hiện nút mạng (Peer Discovery) và quản lý cấu hình thành viên (Membership Management) trong hệ thống được thiết kế theo cấu trúc hai lớp linh hoạt: sử dụng giao thức không hướng kết nối **UDP Broadcast** trong giai đoạn khởi tạo để định vị máy chủ và sử dụng giao thức hướng kết nối **TCP Heartbeat** định kỳ để giám sát trạng thái mạng.

Giải pháp lai này giải quyết triệt để bài toán cấu hình cứng địa chỉ IP (Hardcoding), cho phép các nút mạng tự động thích ứng với sự thay đổi kiến trúc của hạ tầng mạng LAN mà không cần can thiệp vào mã nguồn, đồng thời đảm bảo Bootstrap Server luôn duy trì một bản đồ trạng thái trực tuyến chính xác theo thời gian thực.

4.1. Định vị máy chủ gốc bằng cơ chế UDP Broadcast

Tiến trình tự động dò quét và thiết lập liên kết với Bootstrap Server khi khởi tạo ứng dụng tuân thủ theo kịch bản sau:

1. **Kiểm tra cấu hình:** Khi thành phần Web App cục bộ được kích hoạt, hệ thống sẽ tiến hành kiểm tra các tham số đầu vào. Nếu không tìm thấy tham số chỉ định địa chỉ máy chủ gốc (bootstrap host) truyền vào từ dòng lệnh và biến môi trường hệ thống `BOOTSTRAP_HOST` ở trạng thái trống (Null), ứng dụng sẽ tự động kích hoạt hàm dò quét `discoverBootstrap()`.
2. **Phát tín hiệu quảng bá:** Hàm `discoverBootstrap()` tiến hành khởi tạo một `DatagramSocket` và đóng gói thông điệp định thức văn bản `DISCOVER_BOOTSTRAP`. Gói tin này được phát đồng loạt dưới dạng quảng bá tầng mạng (Network-level Broadcast) tới các địa chỉ quảng bá tiêu chuẩn bao gồm: địa chỉ toàn mạng `255.255.255.255` và các địa chỉ đại diện phân đoạn mạng phổ biến như `192.168.1.255`, `192.168.0.255`, `10.0.0.255` hướng tới cổng dịch vụ quy định `8889`.
3. **Lắng nghe và phản hồi:** Tại máy chủ, Bootstrap Server duy trì một luồng chạy ngầm liên tục gắn chặt (bind) với cổng UDP `8889`. Ngay khi trích xuất được chuỗi ký tự lệnh `DISCOVER_BOOTSTRAP` từ gói tin đến, máy chủ sẽ lập tức phản hồi đơn hướng (Unicast) bản tin mã lệnh `BOOTSTRAP_HERE|ip` (trong đó ip là địa chỉ mạng thực tế của giao tiếp mạng đang hoạt động trên máy chủ) ngược về tọa độ của Peer vừa phát tin.
4. **Thiết lập cấu hình động:** Web App tại Client tiếp nhận dòng dữ liệu phản hồi, bóc tách chuỗi để lấy địa chỉ IP của máy chủ và gán giá trị này cho biến cấu hình toàn cục `bootstrapHost` để phục vụ toàn bộ các kết nối TCP sau đó. Trong trường hợp hết thời gian chờ (Socket Timeout) mà không nhận được bất kỳ phản hồi nào, hệ thống sẽ tự động chuyển đổi về cơ chế dự phòng cục bộ (Fallback mechanism), mặc định máy chủ đang vận hành trên cùng một thực thể vật lý (localhost).



Hình 3.1 : Quy trình Peer Discovery và định vị Bootstrap Server bằng UDP Broadcast

4.2. Duy trì trạng thái trực tuyến bằng cơ chế Nhịp tim (Heartbeat)

Ngay sau khi quá trình xác thực tài khoản tại màn hình đăng nhập hoàn tất, Web App sẽ khởi tạo thực thể kết nối ngang hàng mạng lưới P2PNode và kích hoạt luồng xử lý ngầm thông qua phương thức `startHeartbeat()`.

Cứ định kỳ sau một khoảng thời gian thiết lập cố định là 3 giây, luồng nhịp tim này sẽ tự động đóng gói thông tin định vị mạng hiện tại thành bản tin có định thức:

REGISTER|username-ip:port

và chuyển phát tới cổng TCP 8888 của Bootstrap Server.

Khi tiếp nhận gói tin, Bootstrap Server thực hiện các bước xử lý sau:

- Gọi phương thức nội bộ `UserStore.exists()` để xác thực tính hợp danh của tài khoản trong cơ sở dữ liệu hệ thống.
- Cập nhật thông tin địa chỉ socket mạng (ip:port) của nút vào bảng ánh xạ các nút đang hoạt động (`onlinePeers`).
- Ghi nhận mốc thời gian tương tác vật lý mới nhất của nút đó vào cấu trúc lưu trữ thời gian `peerTimestamps`.

Cơ chế này cho phép máy chủ nắm bắt toàn diện trạng thái "sống" (liveness) của toàn bộ các nút trong mạng một cách liên tục mà không cần duy trì các kết nối TCP dạng giữ lâu (Persistent/Keep-Alive Connections) cồng kềnh, giúp tiết kiệm tài nguyên hệ thống và tối ưu hóa băng thông đường truyền.

4.3. Cơ chế tự động giải phóng nút mạng ngoại tuyến (Timeout)

Để giải quyết bài toán các nút mạng thoát khỏi hệ thống không theo quy trình chuẩn (như tắt ứng dụng đột ngột, sập nguồn thiết bị hoặc mất kết nối mạng vật lý đột xuất), Bootstrap Server triển khai một luồng dọn dẹp bộ nhớ chuyên dụng gọi là `timeout-cleaner`.

Luồng dịch vụ này vận hành độc lập theo chu kỳ 3 giây một lần, quét qua toàn bộ danh sách các mốc thời gian được lưu trữ trong cấu trúc dữ liệu `peerTimestamps`. Tại đây, hệ thống thực hiện phép tính toán đối soát:

$$t_{\text{current}} - t_{\text{last-timestamp}} > \text{TIMEOUT_MS}$$

(Trong đó tham số cấu hình ngưỡng ngắt kết nối `TIMEOUT_MS` được quy định nghiêm ngặt là 10 giây).

Nếu kết quả phép toán vượt quá ngưỡng 10 giây (tương đương với việc nút mạng đã bỏ lỡ quá 3 chu kỳ gửi gói tin nhịp tim liên tiếp), máy chủ sẽ đưa ra kết luận nút mạng đó đã rơi vào trạng thái ngoại tuyến (Offline). Ngay lập tức, hệ thống sẽ thực thi lệnh xóa bỏ toàn bộ các bản ghi liên quan đến định danh người dùng đó khỏi hai cấu trúc quản lý bộ nhớ đệm là `onlinePeers` và `peerTimestamps`.

Nhờ cấu trúc dọn dẹp tự động này, danh sách các Peer trực tuyến trả về cho toàn mạng luôn giữ được sự sạch sẽ, chính xác, loại bỏ hoàn toàn các liên kết "ma" (Stale links) gây lỗi định tuyến khi các Peer thực hiện cuộc gọi kết nối trực tiếp với nhau.

V. XỬ LÝ LỖI VÀ THỬ NGHIỆM HỆ THỐNG

5.1. Cơ chế phòng vệ và xử lý ngoại lệ trong mã nguồn

Để đảm bảo hệ thống vận hành ổn định trong môi trường phân tán (vốn có nhiều bất định về đường truyền và trạng thái nút), mã nguồn được thiết kế với các cơ chế phòng vệ và xử lý ngoại lệ chặt chẽ cho từng tình huống cụ thể:

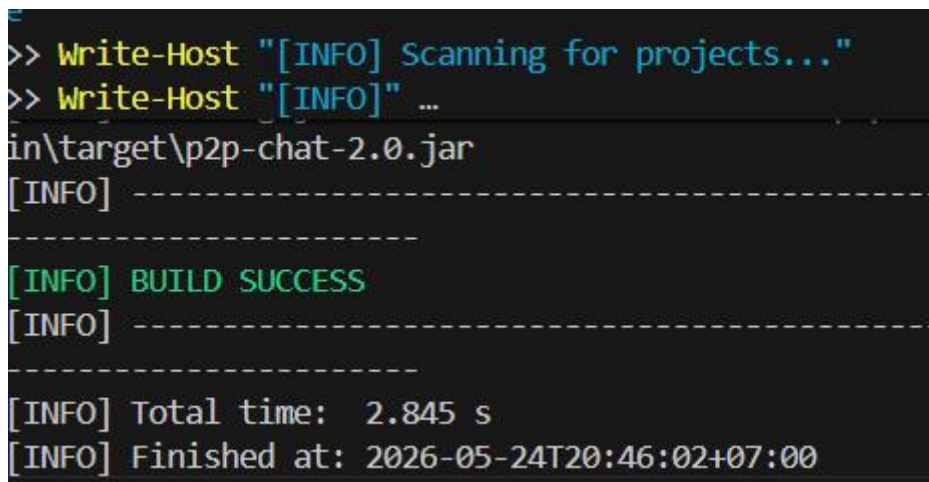
- **Xử lý lỗi định danh khi Đăng ký:** Thành phần UserStore chịu trách nhiệm sàng lọc dữ liệu đầu vào. Hệ thống sẽ trả về mã lỗi INVALID nếu phát hiện tên người dùng trống, chứa các ký tự đặc biệt nằm ngoài tập hợp cho phép (chữ cái, chữ số và dấu gạch dưới), hoặc chuỗi mật khẩu thô có độ dài ngắn hơn 6 ký tự.
 - **Ngăn chặn trùng lặp Tài khoản:** Trước khi thực hiện băm mật khẩu, UserStore tiến hành quét toàn bộ tệp cấu hình users.json. Nếu định danh người dùng đã tồn tại trong hệ thống, tiến trình bị chặn lại và trả về thông báo lỗi USERNAME_TAKEN để bảo vệ tính duy nhất của thực thể.
 - **Xử lý sai cấu trúc xác thực khi Đăng nhập:** Hệ thống sử dụng phương thức BCrypt.verifier() để đối sánh mật khẩu thô do người dùng nhập vào với chuỗi mã băm lưu trong cơ sở dữ liệu. Nếu kết quả kiểm tra không trùng khớp, hệ thống từ chối phiên làm việc và trả về mã lỗi WRONG_PASSWORD.
 - **Xử lý mất kết nối hoặc Peer đích không phản hồi:** Khi phương thức sendDirect thiết lập kết nối TCP Socket tới Peer nhận, hệ thống giới hạn thời gian chờ kết nối (Connect Timeout) là 2000 mili giây. Nếu quá thời gian này hoặc không nhận được tín hiệu phản hồi sau tối đa 02 lần thử lại (Retries), phương thức sẽ trả về giá trị lỗi (false) để báo hiệu đường truyền trực tiếp bị gián đoạn.
 - **Xử lý tình huống Peer ngoại tuyến bằng cơ chế Store-and-Forward:** Ngay khi sendDirect báo lỗi, lớp điều phối App sẽ chuyển sang kịch bản dự phòng bằng cách gọi hàm storeForward. Bản tin lập tức được đẩy lên Bootstrap Server để lưu trữ vào hàng đợi của tệp pending_messages.json và tự động chuyển phát lại khi Peer đích kích hoạt trạng thái trực tuyến.
 - **Giải phóng tài nguyên khi Peer mất mạng đột ngột:** Trong trường hợp một nút mạng bị ngắt kết nối vật lý (mất điện, rút mạng LAN) mà không thực hiện thủ tục đăng xuất tiêu chuẩn, luồng quét ngầm timeout-cleaner trên Bootstrap Server sẽ tự động phát hiện nếu quá 10 giây không nhận được gói tin nhịp tim, từ đó xóa bỏ hoàn toàn dữ liệu của Peer lỗi khỏi bộ nhớ đệm.
 - **Xử lý lỗi định vị Máy chủ gốc (UDP Fallback):** Nếu tiến trình gửi gói tin UDP Broadcast để dò tìm máy chủ không nhận được phản hồi, lớp ứng dụng Web sẽ tự động chuyển đổi về cơ chế mặc định (Fallback), cấu hình đường dẫn hướng về localhost hoặc kiểm tra lại các cấu hình tường minh được thiết lập qua tham số dòng lệnh hoặc biến môi trường BOOTSTRAP_HOST.
 - **Bảo vệ tầng I/O đọc/ghi lịch sử cuộc gọi:** Tại lớp ChatHistoryStore, mọi thao tác tương tác với tệp tin văn bản đều được bao bọc trong các khối lệnh kiểm soát lỗi. Nếu xảy ra ngoại lệ hệ thống (IOException), chương trình sẽ bắt lỗi (catch), thực hiện ghi nhật ký lỗi (Log) và trả về một danh sách rỗng để đảm bảo ứng dụng không bị sập (crash) đột ngột.
 - **Xử lý lỗi giải mã dữ liệu tệp tin (Base64 Exception):** Khi tiếp nhận luồng dữ liệu truyền tệp tại hàm handleIncomingFile, nếu chuỗi ký tự Base64 truyền qua đường text bị lỗi cấu trúc hoặc mất mát gói tin khiến quá trình chuyển đổi về mảng byte thất bại, hệ thống sẽ bắt ngoại lệ, hủy bỏ tiến trình ghi file vào RAM và đẩy log cảnh báo lỗi lên hệ thống quản trị.
-

5.2. Kịch bản thử nghiệm thực tế hệ thống

Quy trình kiểm thử hệ thống được thực hiện tuần tự qua các kịch bản thực tế nhằm đánh giá toàn diện từ giai đoạn đóng gói đến khả năng chịu lỗi của kiến trúc phân tán:

Kịch bản 1: Biên dịch và đóng gói ứng dụng

- **Tác vụ thực hiện:** Kích hoạt dòng lệnh công cụ quản lý dự án mvn package tại thư mục gốc của đồ án.
- **Kết quả mong đợi:** Hệ thống biên dịch thành công toàn bộ mã nguồn, vượt qua các bài kiểm thử và sinh ra hai tệp tin thực thi độc lập tại thư mục đích là target/p2p-bootstrap.jar và target/p2p-app.jar.

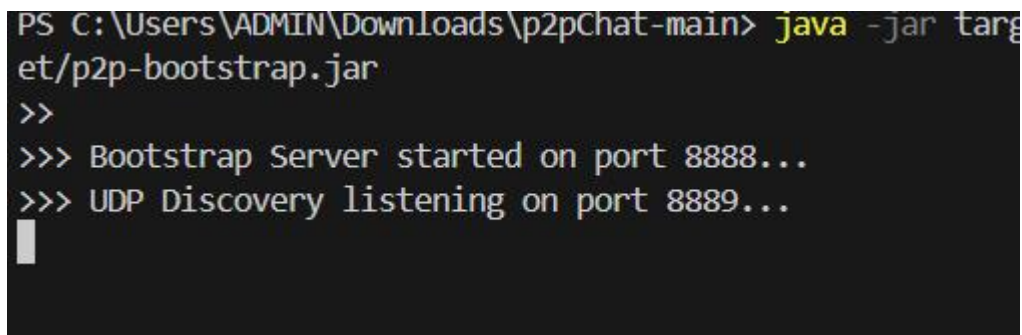


```
>> Write-Host "[INFO] Scanning for projects..."
>> Write-Host "[INFO]" ...
in\target\p2p-chat-2.0.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.845 s
[INFO] Finished at: 2026-05-24T20:46:02+07:00
```

Hình 5.1 : Quá trình biên dịch và đóng gói thành công hệ thống bằng Maven

Kịch bản 2: Khởi chạy Máy chủ điều phối trung tâm

- **Tác vụ thực hiện:** Khởi chạy tệp tin p2p-bootstrap.jar thông qua môi trường thực thi Java.
- **Kết quả mong đợi:** Giao diện dòng lệnh hiển thị thông tin máy chủ đã kích hoạt thành công, bắt đầu lắng nghe các kết nối TCP tại cổng 8888 và các gói tin thăm dò UDP tại cổng 8889.



```
PS C:\Users\ADMIN\Downloads\p2pChat-main> java -jar target/p2p-bootstrap.jar
>>
>>> Bootstrap Server started on port 8888...
>>> UDP Discovery listening on port 8889...
```

Hình 5.2 : Khởi động máy chủ trung tâm Bootstrap Server

Kịch bản 3: Kích hoạt mạng lưới các Nút mạng (Multi-node Activation)

- **Tác vụ thực hiện:** Tiến hành khởi chạy từ 2 đến 3 thực thể ứng dụng p2p-app.jar đồng thời trên hệ thống.
- **Kết quả mong đợi:** Mỗi ứng dụng Web App nhận diện trạng thái và tự động cấp phát một cổng dịch vụ Web tăng dần từ cổng 7000 trở đi (ví dụ: 7000, 7001, 7002) để tránh xung đột tài nguyên.

The screenshot displays three terminal windows within an IDE, each running the command `java -jar target/p2p-app.jar`. The first window shows the application running on port 7000, the second on port 7001, and the third on port 7002. Each window outputs a message: `>>> Giao diện chat: http://localhost:7000` (and so on for the other ports). The IDE interface includes a 'Terminal' tab and a 'Problems' panel on the right.

```

PS C:\Users\ADMIN\Downloads\p2pChat-main> cd C:\Users\ADMIN\Downloads\p2pChat-main
>> java -jar target/p2p-app.jar
>>
for a newer version.).

=====
>>> Giao diện chat: http://localhost:7000
=====

[ ]

java x
\Downloads\p2pChat-main
>> java -jar target/p2p-app.jar
>>

a newer version.).

=====
>>> Giao diện chat: http://localhost:7001
=====

[ ]

Problems Output Debug Console Terminal Ports + v ... | [ ] x
PS C:\Users\ADMIN\Downloads\p2pChat-main> cd C:\Users\ADMIN\Downloads\p2pChat-main
>> java -jar target/p2p-app.jar
>>

[main] INFO io.javalin.Javalin - You are running Javalin 6.1.3 (released March 2, 2024. Your Javalin version is 812 days old. Consider checking for a newer version.).

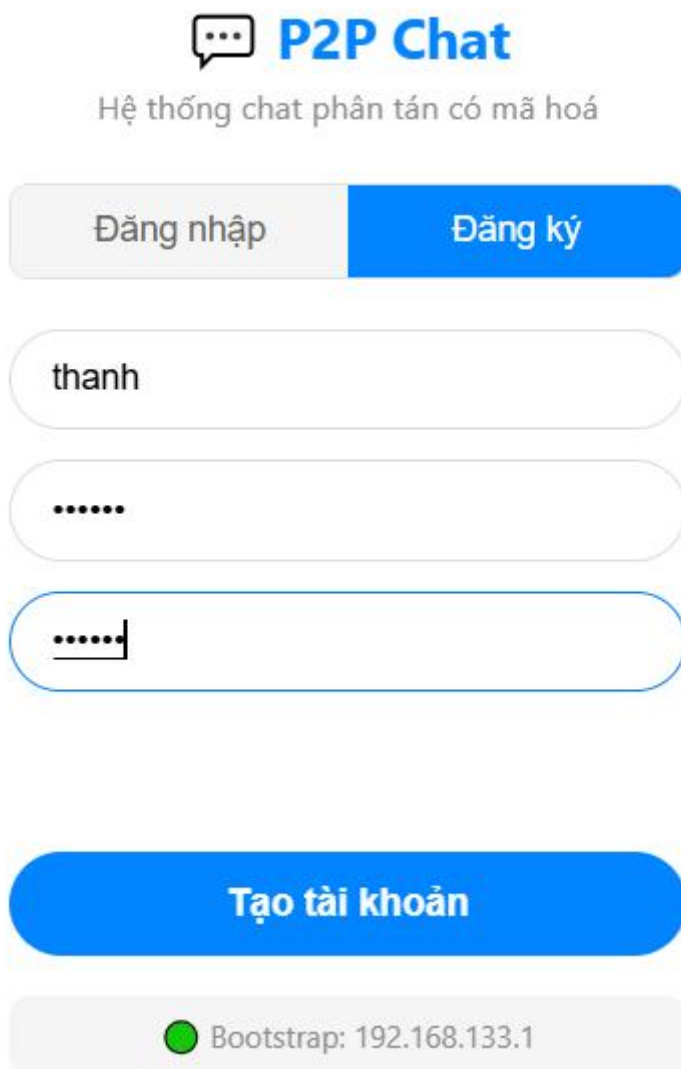
=====
>>> Giao diện chat: http://localhost:7002
=====
  
```

Hình 5.3 : Khởi tạo nhiều Peer Node trên các cổng dịch vụ khác nhau

Kịch bản 4: Kiểm thử tính năng Đăng ký và Đăng nhập

- **Tác vụ thực hiện:** Truy cập giao diện Web, nhập thông tin tài khoản mới để thực hiện đăng ký và tiến hành đăng nhập vào hệ thống.

- **Kết quả mong đợi:** Hệ thống xử lý đăng ký thành công, thông tin tài khoản (đã băm mật khẩu) xuất hiện chính xác trong tệp dữ liệu users.json. Người dùng chuyển hướng vào màn hình điều khiển chính sau khi đăng nhập thành công.



The image shows a web interface for 'P2P Chat'. At the top, there is a logo with a speech bubble icon and the text 'P2P Chat'. Below the logo, it says 'Hệ thống chat phân tán có mã hoá'. There are two buttons: 'Đăng nhập' (Login) and 'Đăng ký' (Register). The 'Đăng ký' button is highlighted in blue. Below the buttons are three input fields. The first field contains the text 'thanh'. The second field contains six dots '.....'. The third field contains six dots '.....' followed by a cursor. Below the input fields is a large blue button labeled 'Tạo tài khoản' (Create account). At the bottom, there is a status bar with a green circle icon and the text 'Bootstrap: 192.168.133.1'.

Hình 5.4.1 : Quá trình đăng ký tài khoản

```

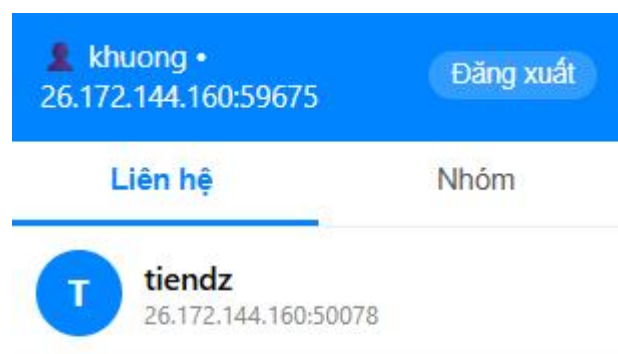
{} users.json > ...
26     "quan1" : {
27         "createdAt" : "2026-05-24T11:51:35.675711400Z"
28     },
29     "thanh" : {
30         "passwordHash" : "$2a$12$jGY6jCx4BKHwF0WvXTDw8eszLQ
31         "createdAt" : "2026-05-24T13:53:14.853161400Z"
32     }
33 }
34 }

```

Hình 5.4.2 : Lưu trữ thông tin user vào file users.json

Kịch bản 5: Kiểm thử cơ chế Khám phá nút mạng (Peer Discovery)

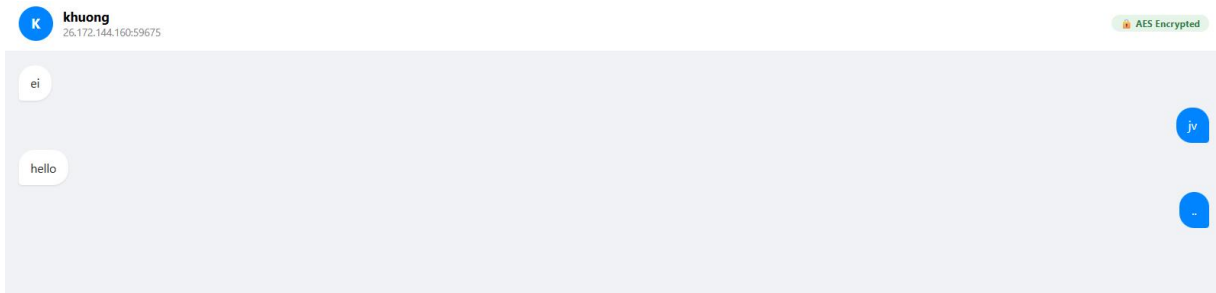
- **Tác vụ thực hiện:** Sau khi nhiều tài khoản thực hiện đăng nhập thành công từ các Web App khác nhau, tiến hành quan sát bảng giám sát trạng thái trực tuyến.
- **Kết quả mong đợi:** Danh sách các Peer trực tuyến trên giao diện của tất cả các tài khoản đồng loạt cập nhật và hiển thị chính xác tên, tọa độ mạng của các thành viên khác vừa tham gia vào hệ thống.



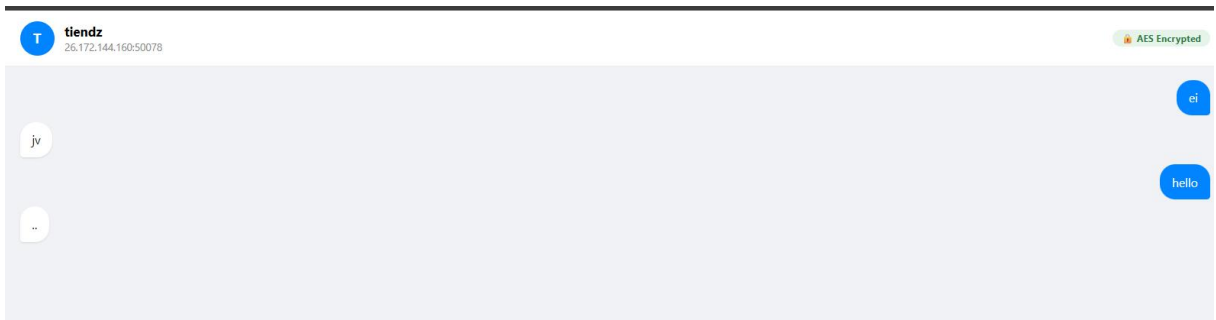
Hình 5.5 : Cơ chế Khám phá nút mạng (Peer Discovery)

Kịch bản 6: Kiểm thử tính năng Trò chuyện riêng trực tiếp (Private Chat)

- **Tác vụ thực hiện:** Sử dụng tài khoản A gửi một thông điệp văn bản trực tiếp đến tài khoản B đang trực tuyến.
- **Kết quả mong đợi:** Thông điệp được truyền thẳng qua kết nối TCP giữa hai nút. Tài khoản B lập tức nhận được tin nhắn thời gian thực, đồng thời cửa sổ chat của tài khoản A hiển thị trạng thái tin nhắn đã gửi thành công sau khi nhận được gói ACK.



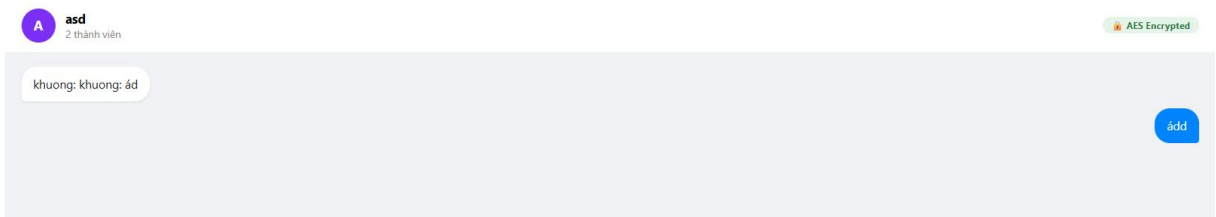
Hình 5.6.1 : Tính năng trò chuyện riêng trực tiếp bên A



Hình 5.6.2 : Tính năng trò chuyện riêng trực tiếp bên B

Kịch bản 7: Kiểm thử tính năng Thiết lập và Trò chuyện nhóm

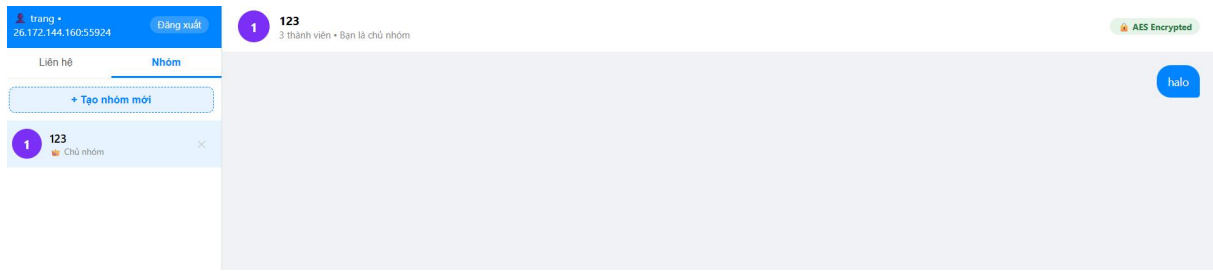
- **Tác vụ thực hiện:** Khởi tạo một nhóm trò chuyện từ tài khoản quản trị, thêm các Peer khác vào nhóm và thực hiện phát đi một thông điệp nhóm.
- **Kết quả mong đợi:** Gói tin mời nhóm ([GROUP_INVITE]) được gửi trực tiếp để cấu hình nhóm cục bộ trên các máy trạm thành viên. Khi gửi tin nhắn nhóm, tất cả các thành viên có trong danh sách nhóm đều nhận được nội dung đồng thời thông qua cơ chế lặp truyền tải đa hướng từ nút nguồn.



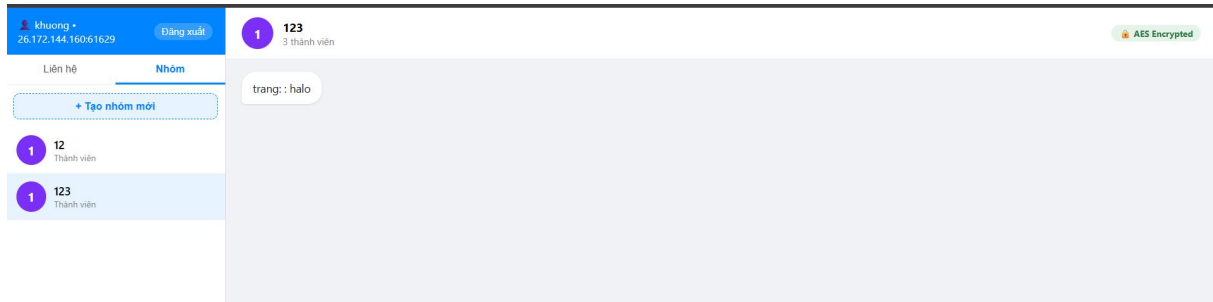
Hình 5.7 : Tính năng thiết lập và trò chuyện nhóm

Kịch bản 8: Kiểm thử tính năng Phát tin quảng bá toàn mạng (Broadcast)

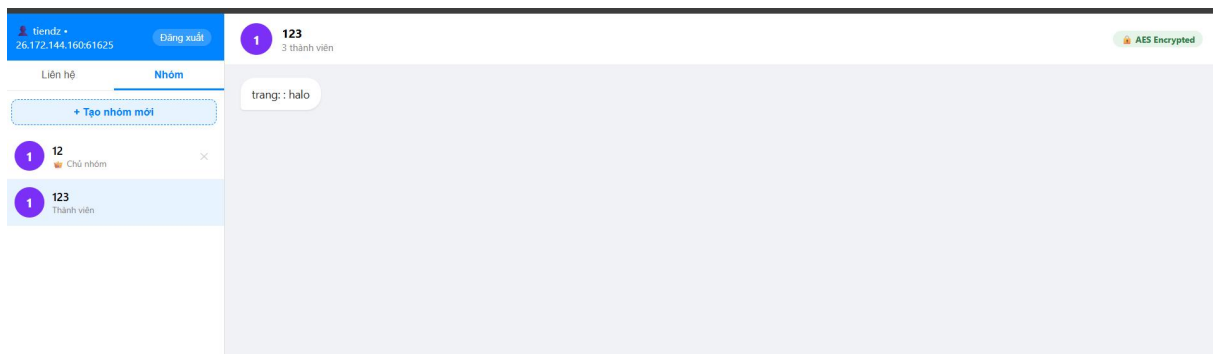
- **Tác vụ thực hiện:** Một Peer bất kỳ chọn tính năng Broadcast và gửi đi một thông điệp chung cho toàn bộ mạng lưới.
- **Kết quả mong đợi:** Nút nguồn tự động duyệt qua danh sách cấu hình, thực hiện gửi song song thông điệp tới toàn bộ các nút đang hiển thị trực tuyến. Tất cả các Peer khác trong hệ thống đều nhận được thông điệp quảng bá này ngay lập tức.



Hình 5.8.1 : Peer A gửi 1 thông điệp Broadcast



Hình 5.8.2 : Peer B nhận thông điệp Broadcast từ Peer A



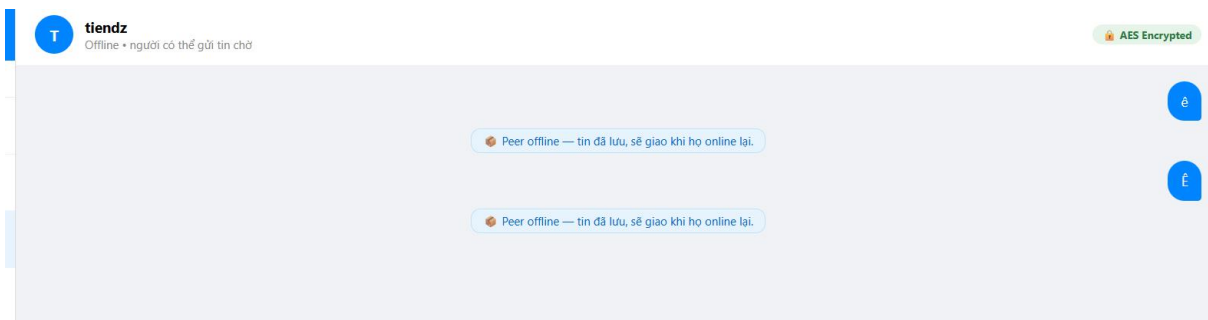
Hình 5.8.3 : Peer C nhận thông điệp Broadcast từ Peer A

Kịch bản 9: Kiểm thử cơ chế Lưu chuyển dự phòng (Store-and-Forward)

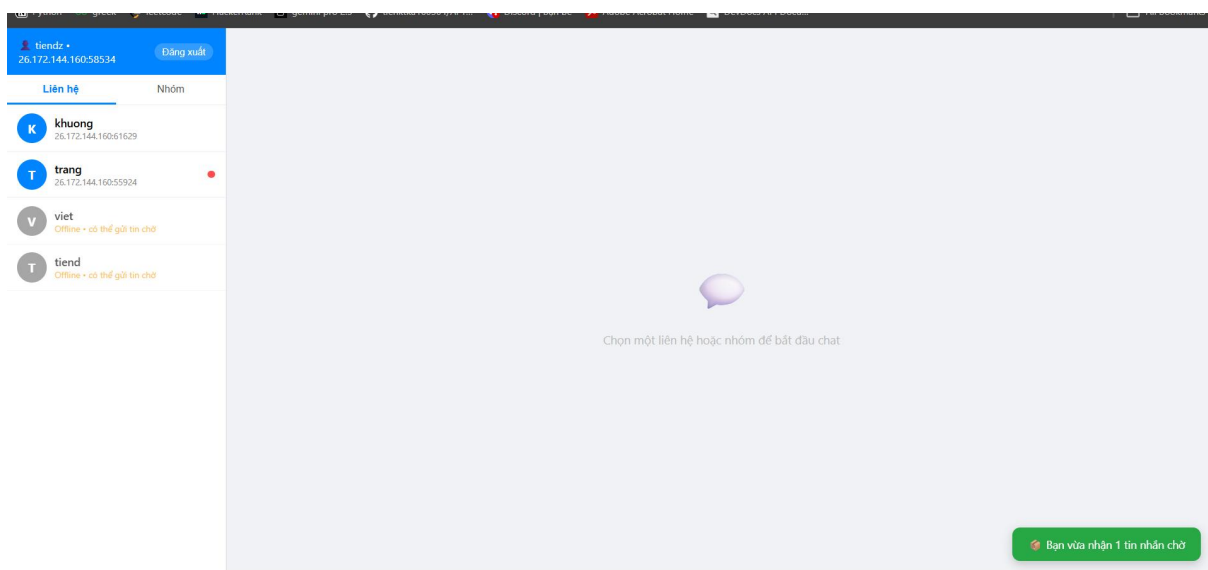
- **Tác vụ thực hiện:** Ngắt kết nối ứng dụng của tài khoản B (chuyển sang ngoại tuyến). Sử dụng tài khoản A để gửi tin nhắn cho B. Sau đó, cho tài khoản B đăng nhập trở lại hệ thống.
- **Kết quả mong đợi:** Khi gửi tin, tài khoản A nhận biết B ngoại tuyến và chuyển tiếp tin nhắn lên Bootstrap Server (Terminal máy chủ báo trạng thái STORED). Ngay khi tài khoản B đăng nhập lại, gói tin phản hồi REGISTER từ máy chủ sẽ giải phóng hàng đợi, bàn giao lại toàn bộ tin nhắn chờ cho B hiển thị lên giao diện.

```
Peer online: tiendz @ 26.172.144.160:61629
Peer r?i m?ng: tiendz
Peer online: khuong @ 26.172.144.160:61629
Peer online: trang @ 26.172.144.160:55924
Peer online: khuong @ 26.172.144.160:61629
Peer online: trang @ 26.172.144.160:55924
>>> L?u tin ch? cho: tiendz
```

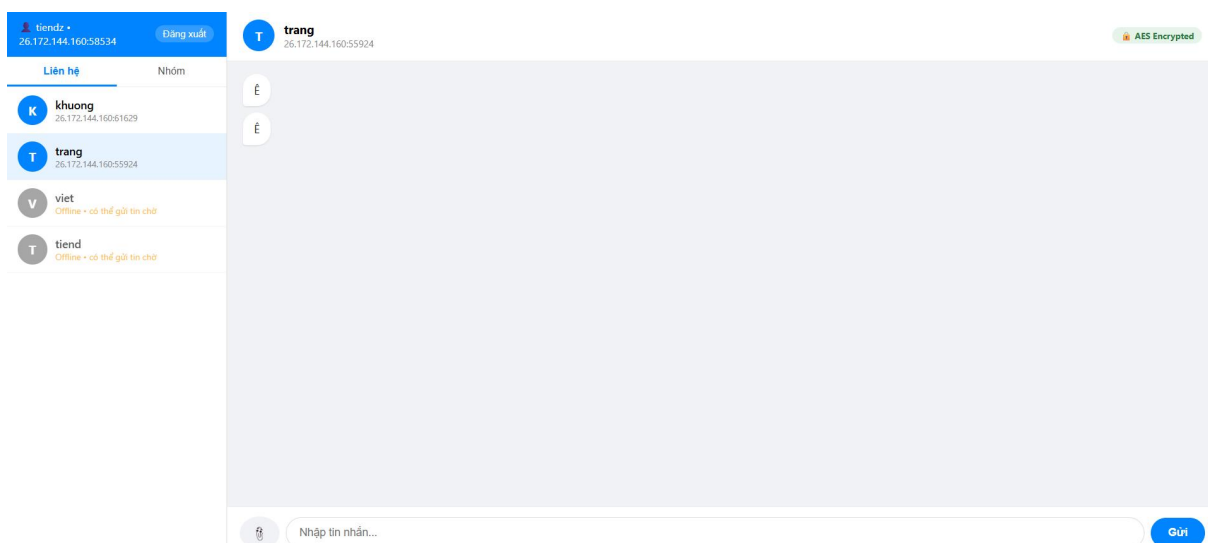

Hình 5.9.1 : Terminal khi Peer A gửi tin nhắn lúc Peer B offline



Hình 5.9.2 : Giao diện Peer A gửi tin nhắn lúc Peer B offline



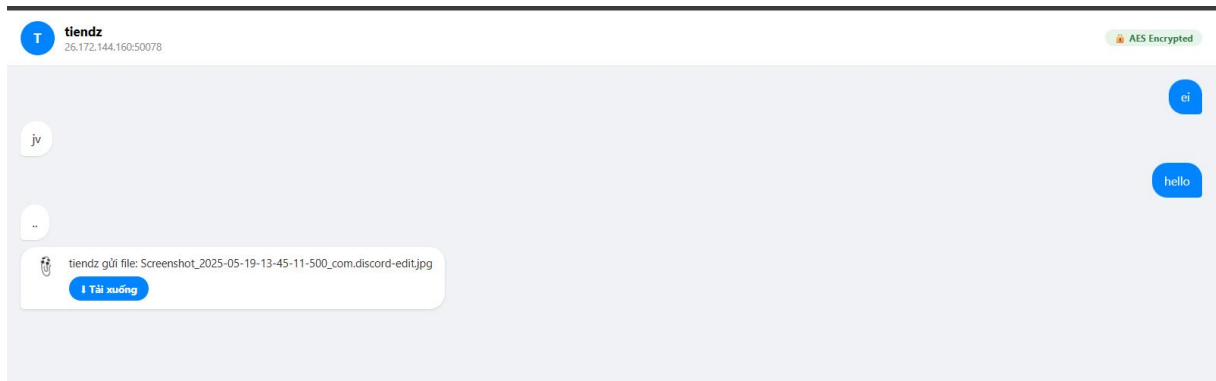
Hình 5.9.3 : Giao diện Peer B nhận được tin nhắn của Peer A lúc online



Hình 5.9.4 : Giao diện Peer B nhận được tin nhắn của Peer A lúc online

Kịch bản 10: Kiểm thử tính năng Truyền tải tệp tin trực tiếp (Direct File Transfer)

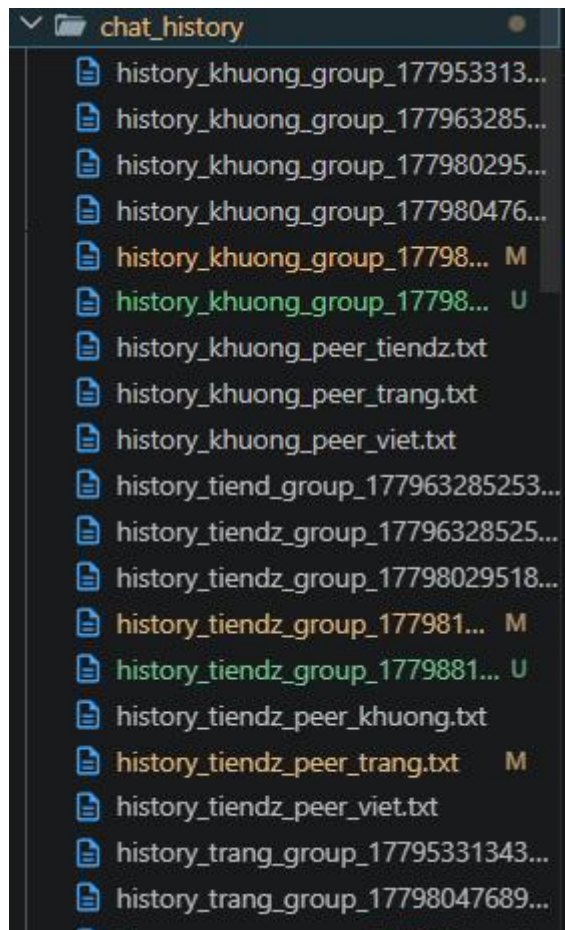
- **Tác vụ thực hiện:** Tài khoản A thực hiện chọn tệp tin từ máy tính vật lý và nhấn lệnh gửi trực tiếp sang tài khoản B.
- **Kết quả mong đợi:** Tệp tin dữ liệu nhị phân được chuyển đổi mã hóa sang chuỗi ký tự Base64 và đẩy qua đường truyền Socket. Tại đầu nhận, tài khoản B hiển thị thanh thông báo nhận tệp tin thành công và có thể nhấn vào nút chức năng để tải tệp tin về lưu trữ cục bộ một cách toàn vẹn.



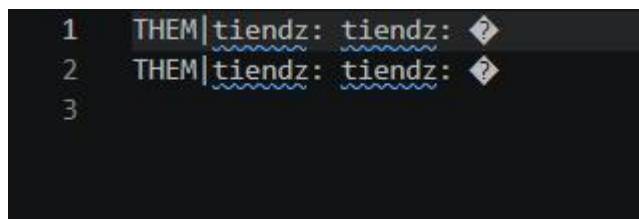
Hình 5.10 : Giao diện tính năng truyền tải tệp tin trực tiếp

Kịch bản 11: Kiểm thử tính năng Lưu trữ lịch sử cuộc hội thoại (Persistence)

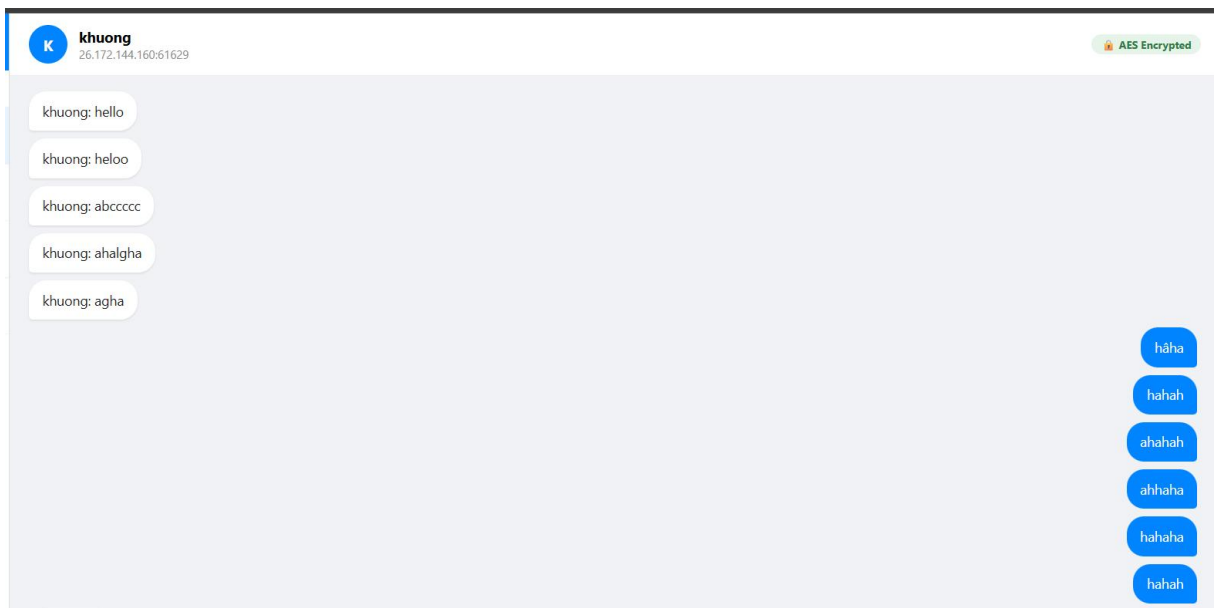
- **Tác vụ thực hiện:** Tiến hành tắt hoàn toàn ứng dụng Web App và khởi động lại, đăng nhập lại vào tài khoản cũ để kiểm tra dữ liệu hội thoại cũ.
- **Kết quả mong đợi:** Hệ thống không bị mất mát dữ liệu trò chuyện. Tầng xử lý ChatHistoryStore tự động quét thư mục chat_history/, đọc lại toàn bộ nội dung trong các tệp văn bản cục bộ và tái dựng đầy đủ lịch sử các cuộc trò chuyện lên khung giao diện của người dùng.



Hình 5.11.1 : Folder nơi lưu trữ lịch sử trò chuyện



Hình 5.11.2 : Cấu trúc lưu trữ lịch sử tin nhắn cục bộ dạng văn bản



Hình 5.11.3 : Giao diện lịch sử trò chuyện

VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đồ án đã nghiên cứu và triển khai thành công mô hình ứng dụng trò chuyện ngang hàng (Hybrid P2P Chat Application) đáp ứng đầy đủ các yêu cầu cốt lõi của môn học Các hệ thống phân tán. Hệ thống đã hiện thực hóa một cơ trúc liên lạc phân tán ổn định bao gồm: cơ chế khám phá nút mạng tự động được hỗ trợ bởi Bootstrap Server, quy trình xác thực định danh an toàn, truyền tải thông điệp văn bản và tệp tin trực tiếp qua giao thức kết nối tin cậy TCP Socket, mã hóa mật mã bảo vệ đường truyền, đồng bộ lịch sử cục bộ và giải pháp lưu chuyển dự phòng Store-and-Forward cho các nút ngoại tuyến.

Thông qua quá trình vận hành thực tế, ứng dụng đã phản ánh rõ nét các đặc trưng cốt lõi của một hệ thống phân tán: từ việc tối ưu hóa luồng giao tiếp mạng, quản lý trạng thái phân tán của các thực thể hoạt động độc lập, phát hiện lỗi ngắt kết nối (Fault Tolerance) cho đến cơ chế đồng bộ dữ liệu mức tối thiểu giữa các nút trạm.

6.1. Ưu điểm của hệ thống

- **Kiến trúc Module hóa tường minh:** Hệ thống phân tách tường minh vai trò của các thực phần: máy chủ định danh trung tâm (Bootstrap Server), lớp điều phối nhúng (Web App) và thực thể mạng lõi (P2PNode). Sự phân tách này giúp mã nguồn dễ dàng bảo trì, nâng cấp và mở rộng chức năng.
- **Tối ưu hóa tài nguyên theo tinh thần Ngang hàng:** Nội dung hội thoại chính và luồng dữ liệu truyền tải tệp tin được thiết lập chạy trực tiếp giữa các Peer với nhau, giải phóng hoàn toàn băng thông hệ thống cho máy chủ trung tâm và triệt tiêu bài toán thắt nút cổ chai (Bottleneck).
- **Giám sát trạng thái mạng linh hoạt:** Triển khai thành công mô hình kiểm soát cấu hình thành viên thông qua chu kỳ nhịp tim (Heartbeat) và tự động ngắt kết nối (Timeout), giúp hệ thống cập nhật bản đồ mạng thời gian thực mà không tiêu tốn tài nguyên giữ kết nối liên tục.
- **Đảm bảo tính tin cậy của thông điệp:** Việc tích hợp giao thức bắt tay xác nhận (ACK) kết hợp kịch bản thử lại tự động (Retry) giúp hệ thống kiểm soát chặt chẽ trạng thái chuyển phát của từng gói tin, từ đó đưa ra quyết định chuyển đổi kịch bản dự phòng Store-and-Forward một cách chính xác.
- **Bền vững hóa dữ liệu ngoại tuyến:** Hàng đợi tin nhắn chờ được đồng bộ xuống tệp tin phẳng pending_messages.json trên máy chủ, đảm bảo dữ liệu không bị mất mát ngay cả trong tình huống Bootstrap Server phải khởi động lại (Restart).
- **Hạ tầng an toàn thông tin tiêu chuẩn:** Ứng dụng tích hợp các chuẩn mật mã học mạnh mẽ, bao gồm thuật toán băm mật khẩu một chiều BCrypt tại tầng quản trị định danh và thuật toán mã hóa đối xứng khối AES/CBC/PKCS5Padding tại tầng truyền tải thông điệp.

6.2. Hạn chế còn tồn tại

- **Tính phụ thuộc vào nút gốc:** Mặc dù luồng dữ liệu chat được phân tán, nhưng Bootstrap Server vẫn đóng vai trò là một điểm yếu chí tử (Single Point of Failure - SPoF). Nếu máy chủ này ngừng hoạt động, các Peer mới sẽ hoàn toàn mất khả năng tham gia mạng và không thể đồng bộ tin nhắn chờ.
 - **Quản lý khóa mật mã tĩnh:** Chuỗi khóa mật mã đối xứng AES hiện đang được thiết lập cố định (Hard-coded) trong mã nguồn ứng dụng, hệ thống chưa triển khai cơ chế trao đổi khóa động hoặc cấp phát khóa riêng biệt cho từng phiên làm việc của các cặp Peer.
-

- **Giới hạn dung lượng truyền tải tệp tin:** Luồng xử lý tệp tin nhận về hiện tại đang lưu trữ trực tiếp trên bộ nhớ đệm RAM của Web App dưới dạng mảng byte. Giải pháp này gây rủi ro tràn bộ nhớ (OutOfMemoryError) và không khả thi khi truyền tải các tệp tin có dung lượng lớn.
- **Luồng xử lý Chat nhóm chưa tối ưu:** Tiến trình gửi tin nhắn nhóm thực chất là tập hợp các kết nối đơn hướng lặp đi lặp lại tới từng thành viên. Hệ thống chưa có cơ chế đồng bộ trạng thái nhóm bền vững (như theo dõi thành viên rời nhóm, phân quyền quản trị) ở mức độ kiến trúc phân tán nâng cao.
- **Rào cản về hạ tầng mạng (NAT Barrier):** Do chưa được tích hợp các kỹ thuật xuyên thủng tường lửa hay các giao thức định tuyến nâng cao (NAT Traversal / STUN / TURN), ứng dụng hiện tại chỉ hoạt động tối ưu trong phạm vi mạng cục bộ LAN hoặc giữa các thiết bị có địa chỉ IP công cộng (Public IP) có khả năng kết nối trực tiếp.

6.3. Hướng phát triển và nâng cấp đề xuất

- **Nâng cấp tầng bảo mật đường truyền:** Tích hợp giao thức bảo mật lớp truyền tải (TLS/SSL) hoặc triển khai thuật toán trao đổi khóa an toàn như Diffie-Hellman để sinh khóa mã hóa AES động, độc lập cho từng phiên hội thoại giữa các cặp Peer.
 - **Tối ưu hóa cơ chế xử lý tệp tin dung lượng lớn:** Chuyển đổi cơ chế lưu trữ file nhận từ RAM xuống phân vùng ổ đĩa vật lý của thiết bị, đồng thời phát triển giải pháp phân rã tệp tin thành các khối dữ liệu nhỏ (Data Chunks) để truyền tải bất đồng bộ, hỗ trợ tính năng tạm dừng và tiếp tục tải file.
 - **Xây dựng Module quản trị nhóm bền vững:** Thiết kế lại cấu trúc quản lý nhóm phân tán, thiết lập quyền hạn thành viên (Admin/Member) và áp dụng các thuật toán đồng thuận phân tán nhẹ để đồng bộ nhất quán trạng thái nhóm giữa các nút trạm.
 - **Mở rộng khả năng kết nối xuyên Internet:** Triển khai tích hợp các giải pháp công nghệ NAT Traversal (như các giao thức STUN/TURN/ICE) hoặc xây dựng cơ chế chuyển tiếp đa bước (Relay/Multi-hop) để hỗ trợ các nút mạng trò chuyện mượt mà qua môi trường Internet mà không bị chặn bởi tường lửa.
 - **Cải tiến tầng Trải nghiệm người dùng (UX/UI):** Tái cấu trúc lại tầng giao diện người dùng dựa trên các framework Frontend chuyên nghiệp (như React hoặc Vue.js); tích hợp thêm cơ chế thông báo trạng thái tin nhắn chi tiết (Đang gửi / Đã tới máy chủ / Đã nhận / Đã đọc) để tăng tính tương tác thời gian thực.
-